React Native Practical Programs

Important Instructions

- Each example below is a separate program.
- Copy only one example at a time into `App.js`.
- These examples use JavaScript, not TypeScript.
- Expo is recommended for beginners.
- Run the project after replacing the code by using:

npx expo start

---

1. Installation and Setup of React Native Environment

Option A: Using Expo

Expo is the recommended starting approach for new React Native projects.

The old global `expo-cli` installation is no longer required; use `npx create-expo-app` instead.

Step 1: Install Node.js

Install the current LTS version of Node.js.

Verify the installation:

```
node --version
```

```
npm --version
```

Step 2: Create an Expo Project

The blank template is convenient for these `App.js` practical programs.

```
npx create-expo-app@latest MyApp --template blank
```

Step 3: Open the Project

```
cd MyApp
```

Step 4: Start the Application

```
npx expo start
```

Step 5: Run the Application

You can run the application using:

- Expo Go on an Android or iOS phone
- Android Emulator

- iOS Simulator on macOS
- Web browser by pressing `w`

To run on a physical phone:

1. Install Expo Go.
2. Connect the computer and phone to the same network.
3. Scan the QR code displayed in the terminal.

Option B: React Native Community CLI

The older command:

```
npx react-native init MyApp
```

should be replaced by the React Native Community CLI command.

Step 1: Install the Required Software

Install:

- Node.js
- Android Studio
- Android SDK
- Java Development Kit
- Visual Studio Code or another code editor

Step 2: Create the Project

```
npx @react-native-community/cli@latest init MyApp
```

Step 3: Enter the Project Folder

cd MyApp

Step 4: Start Metro

Open the first terminal:

npm start

Step 5: Run the Android Application

Keep Metro running and open another terminal:

npm run android

React Native's current documentation uses `npm start` for Metro and `npm run android` to launch the Android application.

---

2. Basic React Native App

This program demonstrates:

- Text

- Image
- Button
- Alert
- Basic styling

```
import React from 'react';

import {

  View,

  Text,

  Image,

  Button,

  Alert,

  StyleSheet,

} from 'react-native';



export default function App() {

  const handlePress = () => {

    Alert.alert(

      'Button Pressed',

      'Welcome to React Native!'

    );

  };
```

```
return (
  <View style={styles.container}>
    <Text style={styles.heading}>
      Hello React Native
    </Text>

    <Text style={styles.description}>
      This is my first React Native application.
    </Text>

    <Image
      source={{
        uri: 'https://reactnative.dev/img/tiny_logo.png',
      }}
      style={styles.logo}
      resizeMode="contain"
    />

    <View style={styles.buttonContainer}>
```

```
        <Button
          title="Click Me"
          onPress={handlePress}
        />
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 24,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#f5f5f5',
  },

  heading: {
    fontSize: 28,
```

```
    fontWeight: 'bold',
    marginBottom: 10,
  },

  description: {
    fontSize: 16,
    textAlign: 'center',
    marginBottom: 25,
  },

  logo: {
    width: 120,
    height: 120,
    marginBottom: 25,
  },

  buttonContainer: {
    width: 180,
  },
});
```

---

3. UI Design Using Core Components

This program demonstrates:

- `View`
- `Text`
- `ScrollView`
- Reusable item cards
- Vertical scrolling

```
import React from 'react';

import {

  View,

  Text,

  ScrollView,

  StyleSheet,

} from 'react-native';



const subjects = [
```

```
{
  id: 1,
  name: 'React Native',
  description: 'Mobile application development',
},
{
  id: 2,
  name: 'JavaScript',
  description: 'Programming language for React',
},
{
  id: 3,
  name: 'Firebase',
  description: 'Backend and authentication service',
},
{
  id: 4,
  name: 'API Integration',
  description: 'Connecting applications with servers',
},
```

```
  {
    id: 5,
    name: 'Navigation',
    description: 'Moving between application screens',
  },
];

export default function App() {
  return (
    <ScrollView
      style={styles.scrollView}
      contentContainerStyle={styles.content}
    >
      <Text style={styles.heading}>
        React Native Subjects
      </Text>

      {subjects.map((subject) => (
        <View
          key={subject.id}
```

```
          style={styles.card}
        >
          <Text style={styles.title}>
            {subject.name}
          </Text>

          <Text style={styles.description}>
            {subject.description}
          </Text>
        </View>
      ))}
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  scrollView: {
    flex: 1,
    backgroundColor: '#f4f6f8',
  },
```

```
content: {
  padding: 20,
  paddingTop: 60,
},

heading: {
  fontSize: 27,
  fontWeight: 'bold',
  marginBottom: 20,
},

card: {
  padding: 20,
  marginBottom: 15,
  backgroundColor: '#ffffff',
  borderRadius: 12,
  elevation: 3,
},
```

```
  title: {
    fontSize: 20,
    fontWeight: 'bold',
    marginBottom: 6,
  },

  description: {
    fontSize: 15,
    lineHeight: 22,
  },
});
```

---

4. Styling Using StyleSheet and Flexbox

This program demonstrates:

- `StyleSheet`
- `flex`
- `flexDirection`

- `justifyContent`
- `alignItems`
- Responsive box layout

```
import React from 'react';

import {

  View,

  Text,

  StyleSheet,

} from 'react-native';



export default function App() {

  return (

    <View style={styles.container}>

      <Text style={styles.heading}>

        Flexbox Layout

      </Text>



      <View style={styles.row}>

        <View style={styles.box}>

          <Text style={styles.boxText}>1</Text>

        </View>
```

```
      <View style={styles.box}>
        <Text style={styles.boxText}>2</Text>
      </View>

      <View style={styles.box}>
        <Text style={styles.boxText}>3</Text>
      </View>
    </View>

    <View style={styles.informationBox}>
      <Text style={styles.informationText}>
        The boxes are arranged horizontally using
        flexDirection: 'row'.
      </Text>
    </View>
  </View>
 );
}
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    justifyContent: 'center',
    backgroundColor: '#f5f5f5',
  },

  heading: {
    fontSize: 28,
    fontWeight: 'bold',
    textAlign: 'center',
    marginBottom: 30,
  },

  row: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
  },
```

```
box: {
  width: 90,
  height: 90,
  justifyContent: 'center',
  alignItems: 'center',
  backgroundColor: '#2563eb',
  borderRadius: 12,
},

boxText: {
  fontSize: 26,
  fontWeight: 'bold',
  color: '#ffffff',
},

informationBox: {
  marginTop: 30,
  padding: 18,
  backgroundColor: '#ffffff',
```

```
    borderRadius: 10,
  },

  informationText: {
    fontSize: 16,
    lineHeight: 24,
    textAlign: 'center',
  },
});
```

---

5. User Input Form Creation

This program accepts:

- Name
- Email
- Mobile number

It also performs basic validation before displaying the submitted information.

```
import React, { useState } from 'react';
```

```
import {
  View,
  Text,
  TextInput,
  Pressable,
  Alert,
  StyleSheet,
  KeyboardAvoidingView,
  Platform,
} from 'react-native';

export default function App() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [mobile, setMobile] = useState('');

  const handleSubmit = () => {
    if (!name.trim()) {
      Alert.alert('Validation Error', 'Please enter your name.');
      return;
```

```
}

if (!email.trim()) {
  Alert.alert('Validation Error', 'Please enter your email.');
  return;
}

if (!mobile.trim()) {
  Alert.alert(
    'Validation Error',
    'Please enter your mobile number.'
  );
  return;
}

Alert.alert(
  'Form Submitted',
  `Name: ${name}\nEmail: ${email}\nMobile: ${mobile}`
);
```

```
  setName('');
  setEmail('');
  setMobile('');
};

return (
  <KeyboardAvoidingView
    style={styles.container}
    behavior={Platform.OS === 'ios' ? 'padding' : undefined}
  >
    <View style={styles.form}>
      <Text style={styles.heading}>
        Student Registration
      </Text>

      <Text style={styles.label}>Name</Text>

      <TextInput
        style={styles.input}
        placeholder="Enter your name"
```

```
  value={name}
  onChangeText={setName}
  autoCapitalize="words"
/>

<Text style={styles.label}>Email</Text>

<TextInput
  style={styles.input}
  placeholder="Enter your email"
  value={email}
  onChangeText={setEmail}
  keyboardType="email-address"
  autoCapitalize="none"
/>

<Text style={styles.label}>Mobile Number</Text>

<TextInput
  style={styles.input}
```

```
        placeholder="Enter mobile number"
        value={mobile}
        onChangeText={setMobile}
        keyboardType="phone-pad"
        maxLength={10}
      />

      <Pressable
        style={styles.button}
        onPress={handleSubmit}
      >
        <Text style={styles.buttonText}>
          Submit Form
        </Text>
      </Pressable>
    </View>
  </KeyboardAvoidingView>
 );
}
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    padding: 20,
    backgroundColor: '#eef2f7',
  },

  form: {
    padding: 22,
    backgroundColor: '#ffffff',
    borderRadius: 14,
    elevation: 4,
  },

  heading: {
    fontSize: 25,
    fontWeight: 'bold',
    textAlign: 'center',
    marginBottom: 25,
```

```
},

label: {
  fontSize: 15,
  fontWeight: '600',
  marginBottom: 7,
},

input: {
  height: 48,
  paddingHorizontal: 14,
  marginBottom: 18,
  borderWidth: 1,
  borderColor: '#cbd5e1',
  borderRadius: 8,
},

button: {
  paddingVertical: 14,
  alignItems: 'center',
```

```
    backgroundColor: '#2563eb',
    borderRadius: 8,
  },

  buttonText: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#ffffff',
  },
});
```

---

6. Props and State in Functional Components

In this example:

- The parent component stores data in state.
- The parent passes the data to the child using props.
- The child displays the received data.

```
import React, { useState } from 'react';
```

```
import {
  View,
  Text,
  Button,
  StyleSheet,
} from 'react-native';

function Child({ message, studentName }) {
  return (
    <View style={styles.childContainer}>
      <Text style={styles.childHeading}>
        Child Component
      </Text>

      <Text style={styles.message}>
        {message}
      </Text>

      <Text style={styles.student}>
        Student: {studentName}
```

```
      </Text>
    </View>
  );
}

export default function App() {
  const [message, setMessage] = useState(
    'Hello from the Parent Component'
  );

  const changeMessage = () => {
    setMessage('The parent state has been updated!');
  };

  return (
    <View style={styles.container}>
      <Text style={styles.heading}>
        Props and State Example
      </Text>
```

```
      <Child
        message={message}
        studentName="John"
      />

      <Button
        title="Change Message"
        onPress={changeMessage}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 24,
    justifyContent: 'center',
    backgroundColor: '#f1f5f9',
  },
```

```
heading: {
  fontSize: 27,
  fontWeight: 'bold',
  textAlign: 'center',
  marginBottom: 25,
},

childContainer: {
  padding: 22,
  marginBottom: 20,
  backgroundColor: '#ffffff',
  borderRadius: 12,
  elevation: 3,
},

childHeading: {
  fontSize: 20,
  fontWeight: 'bold',
  marginBottom: 12,
```

```
  },

  message: {
    fontSize: 17,
    marginBottom: 10,
  },

  student: {
    fontSize: 16,
    fontWeight: '600',
  },
});
```

---

7. Counter App Using State

This version contains:

- Increase
- Decrease
- Reset

- Protection against negative values

```
import React, { useState } from 'react';
import {
  View,
  Text,
  Pressable,
  StyleSheet,
} from 'react-native';

export default function App() {
  const [count, setCount] = useState(0);

  const increaseCount = () => {
    setCount((previousCount) => previousCount + 1);
  };

  const decreaseCount = () => {
    setCount((previousCount) =>
      previousCount > 0 ? previousCount - 1 : 0
    );
```

```
};

const resetCount = () => {
  setCount(0);
};

return (
  <View style={styles.container}>
    <Text style={styles.heading}>
      Counter Application
    </Text>

    <View style={styles.counterBox}>
      <Text style={styles.counter}>
        {count}
      </Text>
    </View>

    <View style={styles.buttonRow}>
      <Pressable
```

```
    style={styles.button}
    onPress={decreaseCount}
  >
    <Text style={styles.buttonText}>-</Text>
  </Pressable>

  <Pressable
    style={styles.button}
    onPress={increaseCount}
  >
    <Text style={styles.buttonText}>+</Text>
  </Pressable>
</View>

<Pressable
  style={styles.resetButton}
  onPress={resetCount}
>
  <Text style={styles.resetButtonText}>
    Reset Counter
```

```
        </Text>
      </Pressable>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 25,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#f8fafc',
  },

  heading: {
    fontSize: 28,
    fontWeight: 'bold',
    marginBottom: 30,
  },
```

```
counterBox: {
  width: 160,
  height: 160,
  justifyContent: 'center',
  alignItems: 'center',
  backgroundColor: '#ffffff',
  borderRadius: 80,
  elevation: 5,
},

counter: {
  fontSize: 58,
  fontWeight: 'bold',
},

buttonRow: {
  flexDirection: 'row',
  marginTop: 30,
  gap: 25,
```

```
},

button: {
  width: 70,
  height: 55,
  justifyContent: 'center',
  alignItems: 'center',
  backgroundColor: '#2563eb',
  borderRadius: 10,
},

buttonText: {
  fontSize: 32,
  fontWeight: 'bold',
  color: '#ffffff',
},

resetButton: {
  marginTop: 25,
  paddingVertical: 14,
```

```
    paddingHorizontal: 30,
    backgroundColor: '#dc2626',
    borderRadius: 8,
  },

  resetButtonText: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#ffffff',
  },
});
```

---

8. Stack Navigation Implementation

Native Stack Navigation uses the native navigation APIs of Android and iOS and generally provides smoother transitions than the JavaScript-based stack navigator.

Installation

For an Expo project:

npm install @react-navigation/native

npm install @react-navigation/native-stack

npx expo install react-native-screens react-native-safe-area-context

Complete Program

```
import React from 'react';

import {

  View,

  Text,

  Button,

  StyleSheet,

} from 'react-native';



import {

  NavigationContainer,

} from '@react-navigation/native';
```

```
import {
  createNativeStackNavigator,
} from '@react-navigation/native-stack';

const Stack = createNativeStackNavigator();

function HomeScreen({ navigation }) {
  return (
    <View style={styles.container}>
      <Text style={styles.heading}>
        Home Screen
      </Text>

      <Text style={styles.description}>
        This is the first screen of the application.
      </Text>

      <Button
        title="Go to Details"
        onPress={() => navigation.navigate('Details')}
```

```
      />
    </View>
  );
}

function DetailsScreen({ navigation }) {
  return (
    <View style={styles.container}>
      <Text style={styles.heading}>
        Details Screen
      </Text>

      <Text style={styles.description}>
        You have successfully navigated to another screen.
      </Text>

      <Button
        title="Go Back"
        onPress={() => navigation.goBack()}
      />
```

```
    </View>
  );
}

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator
        initialRouteName="Home"
        screenOptions={{
          headerTitleAlign: 'center',
        }}
      >
        <Stack.Screen
          name="Home"
          component={HomeScreen}
          options={{
            title: 'Home',
          }}
        />
```

```
        <Stack.Screen
          name="Details"
          component={DetailsScreen}
          options={{
            title: 'Details',
          }}
        />
      </Stack.Navigator>
    </NavigationContainer>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 25,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#f8fafc',
```

```
  },

  heading: {
    fontSize: 28,
    fontWeight: 'bold',
    marginBottom: 12,
  },

  description: {
    fontSize: 16,
    lineHeight: 24,
    textAlign: 'center',
    marginBottom: 25,
  },
});
```

---

9. Bottom Tab Navigation

Bottom Tab Navigation allows the user to switch between application screens using a tab bar at the bottom.

Installation

npm install @react-navigation/native

npm install @react-navigation/bottom-tabs

npx expo install react-native-screens react-native-safe-area-context

Complete Program

```
import React from 'react';
import {
  View,
  Text,
  StyleSheet,
} from 'react-native';


import {
  NavigationContainer,
```

```
} from '@react-navigation/native';

import {
  createBottomTabNavigator,
} from '@react-navigation/bottom-tabs';

const Tab = createBottomTabNavigator();

function HomeScreen() {
  return (
    <View style={styles.container}>
      <Text style={styles.icon}>🏠</Text>
      <Text style={styles.heading}>Home Screen</Text>
      <Text style={styles.description}>
        Welcome to the home section.
      </Text>
    </View>
  );
}
```

```
function ProfileScreen() {
  return (
    <View style={styles.container}>
      <Text style={styles.icon}> 👤 </Text>
      <Text style={styles.heading}>Profile Screen</Text>
      <Text style={styles.description}>
        This section displays user information.
      </Text>
    </View>
  );
}

function SettingsScreen() {
  return (
    <View style={styles.container}>
      <Text style={styles.icon}> ⚙️ </Text>
      <Text style={styles.heading}>Settings Screen</Text>
      <Text style={styles.description}>
        Manage application settings here.
      </Text>
```

```
    </View>
  );
}


export default function App() {
  return (
    <NavigationContainer>
      <Tab.Navigator
        screenOptions={{
          headerTitleAlign: 'center',
          tabBarLabelStyle: {
            fontSize: 13,
          },
          tabBarStyle: {
            height: 65,
            paddingBottom: 8,
          },
        }}
      >
        <Tab.Screen
```

```
          name="Home"
          component={HomeScreen}
        />

        <Tab.Screen
          name="Profile"
          component={ProfileScreen}
        />

        <Tab.Screen
          name="Settings"
          component={SettingsScreen}
        />
      </Tab.Navigator>
    </NavigationContainer>
  );
}

const styles = StyleSheet.create({
  container: {
```

```
  flex: 1,
  padding: 25,
  justifyContent: 'center',
  alignItems: 'center',
  backgroundColor: '#f8fafc',
},

icon: {
  fontSize: 55,
  marginBottom: 15,
},

heading: {
  fontSize: 27,
  fontWeight: 'bold',
  marginBottom: 10,
},

description: {
  fontSize: 16,
```

```
    textAlign: 'center',
  },
});
```

---

10. Passing Data Between Screens

Data can be passed as the second argument of `navigation.navigate()` and received through `route.params`.

Installation

npm install @react-navigation/native

npm install @react-navigation/native-stack

npx expo install react-native-screens react-native-safe-area-context

Complete Program

```
import React, { useState } from 'react';
import {
  View,
  Text,
```

```
  TextInput,
  Button,
  StyleSheet,
} from 'react-native';

import {
  NavigationContainer,
} from '@react-navigation/native';

import {
  createNativeStackNavigator,
} from '@react-navigation/native-stack';

const Stack = createNativeStackNavigator();

function HomeScreen({ navigation }) {
  const [name, setName] = useState('');
  const [course, setCourse] = useState('');

  const openDetails = () => {
```

```
  navigation.navigate('Details', {
    studentName: name || 'Not provided',
    courseName: course || 'Not provided',
  });
};

return (
  <View style={styles.container}>
    <Text style={styles.heading}>
      Enter Student Details
    </Text>

    <TextInput
      style={styles.input}
      placeholder="Enter student name"
      value={name}
      onChangeText={setName}
    />

    <TextInput
```

```
        style={styles.input}
        placeholder="Enter course name"
        value={course}
        onChangeText={setCourse}
      />

      <Button
        title="Open Details"
        onPress={openDetails}
      />
    </View>
  );
}

function DetailsScreen({ route }) {
  const {
    studentName,
    courseName,
  } = route.params;
```

```
return (
  <View style={styles.container}>
    <Text style={styles.heading}>
      Student Information
    </Text>

    <View style={styles.card}>
      <Text style={styles.label}>
        Student Name
      </Text>

      <Text style={styles.value}>
        {studentName}
      </Text>

      <Text style={styles.label}>
        Course
      </Text>

      <Text style={styles.value}>
```

```
        {courseName}
      </Text>
    </View>
  </View>
 );
}

export default function App() {
 return (
  <NavigationContainer>
   <Stack.Navigator>
    <Stack.Screen
     name="Home"
     component={HomeScreen}
     options={{
      title: 'Student Form',
     }}
    />

    <Stack.Screen
```

```
          name="Details"
          component={DetailsScreen}
          options={{
            title: 'Student Details',
          }}
        />
      </Stack.Navigator>
    </NavigationContainer>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 25,
    justifyContent: 'center',
    backgroundColor: '#f8fafc',
  },

  heading: {
```

```
  fontSize: 27,
  fontWeight: 'bold',
  textAlign: 'center',
  marginBottom: 25,
},

input: {
  height: 50,
  paddingHorizontal: 14,
  marginBottom: 16,
  backgroundColor: '#ffffff',
  borderWidth: 1,
  borderColor: '#cbd5e1',
  borderRadius: 8,
},

card: {
  padding: 22,
  backgroundColor: '#ffffff',
  borderRadius: 12,
```

```
    elevation: 3,
  },

  label: {
    fontSize: 15,
    fontWeight: 'bold',
    marginBottom: 5,
  },

  value: {
    fontSize: 18,
    marginBottom: 20,
  },
});
```

---

11. Global State Using Context API

Context API makes data available to multiple components without passing props manually through every component.

```
import React, {
  createContext,
  useContext,
  useState,
} from 'react';

import {
  View,
  Text,
  TextInput,
  Pressable,
  StyleSheet,
} from 'react-native';

const UserContext = createContext(null);

function Profile() {
  const {
    user,
    updateUser,
```

```
} = useContext(UserContext);


const [newName, setNewName] = useState('');


const handleUpdate = () => {
  if (newName.trim()) {
    updateUser(newName.trim());
    setNewName('');
  }
};


return (
  <View style={styles.card}>
    <Text style={styles.cardHeading}>
      Profile Component
    </Text>


    <Text style={styles.welcomeText}>
      Welcome, {user}
    </Text>
```

```
      <TextInput
        style={styles.input}
        placeholder="Enter a new name"
        value={newName}
        onChangeText={setNewName}
      />

      <Pressable
        style={styles.button}
        onPress={handleUpdate}
      >
        <Text style={styles.buttonText}>
          Update Global User
        </Text>
      </Pressable>
    </View>
  );
}
```

```
function Dashboard() {
  const {
    user,
  } = useContext(UserContext);

  return (
    <View style={styles.card}>
      <Text style={styles.cardHeading}>
        Dashboard Component
      </Text>

      <Text style={styles.welcomeText}>
        Current logged-in user: {user}
      </Text>
    </View>
  );
}

export default function App() {
  const [user, setUser] = useState('John');
```

```
const updateUser = (name) => {
  setUser(name);
};

return (
  <UserContext.Provider
    value={{
      user,
      updateUser,
    }}
  >
    <View style={styles.container}>
      <Text style={styles.heading}>
        Context API Example
      </Text>

      <Profile />
      <Dashboard />
    </View>
```

```
    </UserContext.Provider>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    paddingTop: 60,
    backgroundColor: '#f1f5f9',
  },

  heading: {
    fontSize: 28,
    fontWeight: 'bold',
    textAlign: 'center',
    marginBottom: 25,
  },

  card: {
```

```
  padding: 20,
  marginBottom: 18,
  backgroundColor: '#ffffff',
  borderRadius: 12,
  elevation: 3,
},

cardHeading: {
  fontSize: 20,
  fontWeight: 'bold',
  marginBottom: 12,
},

welcomeText: {
  fontSize: 17,
  marginBottom: 15,
},

input: {
  height: 48,
```

```
    paddingHorizontal: 12,
    marginBottom: 14,
    borderWidth: 1,
    borderColor: '#cbd5e1',
    borderRadius: 8,
  },

  button: {
    padding: 13,
    alignItems: 'center',
    backgroundColor: '#2563eb',
    borderRadius: 8,
  },

  buttonText: {
    fontSize: 15,
    fontWeight: 'bold',
    color: '#ffffff',
  },
});
```

---

12. API Integration Using Fetch

This program includes:

- `useEffect`
- Fetch API
- Loading state
- Error handling
- `FlatList`
- Reload button

```
import React, {
  useEffect,
  useState,
} from 'react';


import {
  View,
  Text,
  FlatList,
  ActivityIndicator,
```

```
  Pressable,
  StyleSheet,
} from 'react-native';

const API_URL =
  'https://jsonplaceholder.typicode.com/posts';

export default function App() {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  const fetchPosts = async () => {
    try {
      setLoading(true);
      setError('');

      const response = await fetch(API_URL);

      if (!response.ok) {
```

```
        throw new Error(
          `Server returned status ${response.status}`
        );
      }

      const data = await response.json();

      setPosts(data.slice(0, 15));
    } catch (fetchError) {
      setError(
        fetchError.message ||
          'Unable to load the posts.'
      );
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchPosts();
```

```
}, []);



if (loading) {
  return (
    <View style={styles.centerContainer}>
      <ActivityIndicator size="large" />

      <Text style={styles.loadingText}>
        Loading posts...
      </Text>
    </View>
  );
}


if (error) {
  return (
    <View style={styles.centerContainer}>
      <Text style={styles.errorHeading}>
        Unable to Load Data
      </Text>
```

```
      <Text style={styles.errorText}>
        {error}
      </Text>

      <Pressable
        style={styles.reloadButton}
        onPress={fetchPosts}
      >
        <Text style={styles.reloadButtonText}>
          Try Again
        </Text>
      </Pressable>
    </View>
  );
}

return (
  <View style={styles.container}>
    <Text style={styles.heading}>
```

```
  API Posts

</Text>



<FlatList

  data={posts}

  keyExtractor={(item) =>

    item.id.toString()

  }

  showsVerticalScrollIndicator={false}

  renderItem={({ item }) => (

    <View style={styles.card}>

      <Text style={styles.postNumber}>

        Post #{item.id}

      </Text>



      <Text style={styles.title}>

        {item.title}

      </Text>



      <Text style={styles.body}>
```

```
            {item.body}
          </Text>
        </View>
      )}
    />
  </View>
 );
}


const styles = StyleSheet.create({
 container: {
  flex: 1,
  paddingHorizontal: 18,
  paddingTop: 55,
  backgroundColor: '#f1f5f9',
 },

 centerContainer: {
  flex: 1,
  padding: 25,
```

```
  justifyContent: 'center',
  alignItems: 'center',
},

heading: {
  fontSize: 28,
  fontWeight: 'bold',
  marginBottom: 18,
},

loadingText: {
  fontSize: 16,
  marginTop: 14,
},

card: {
  padding: 18,
  marginBottom: 14,
  backgroundColor: '#ffffff',
  borderRadius: 12,
```

```
  elevation: 2,
},

postNumber: {
  fontSize: 13,
  fontWeight: 'bold',
  marginBottom: 7,
},

title: {
  fontSize: 18,
  fontWeight: 'bold',
  textTransform: 'capitalize',
  marginBottom: 8,
},

body: {
  fontSize: 15,
  lineHeight: 22,
},
```

```
errorHeading: {
  fontSize: 24,
  fontWeight: 'bold',
  marginBottom: 12,
},

errorText: {
  fontSize: 16,
  textAlign: 'center',
  marginBottom: 20,
},

reloadButton: {
  paddingVertical: 13,
  paddingHorizontal: 25,
  backgroundColor: '#2563eb',
  borderRadius: 8,
},
```

```
  reloadButtonText: {
    fontWeight: 'bold',
    color: '#ffffff',
  },
});
```

---

13. Form Validation Using Formik and Yup

Formik manages the form state, while Yup defines the validation rules.

Installation

npm install formik yup

Complete Program

```
import React from 'react';

import {
```

```
  View,
  Text,
  TextInput,
  Pressable,
  Alert,
  StyleSheet,
  KeyboardAvoidingView,
  Platform,
} from 'react-native';

import {
  Formik,
} from 'formik';

import * as Yup from 'yup';

const validationSchema = Yup.object({
  name: Yup.string()
    .trim()
    .min(3, 'Name must contain at least 3 characters.')
```

```
    .required('Name is required.'),

  email: Yup.string()
    .trim()
    .email('Enter a valid email address.')
    .required('Email is required.'),

  password: Yup.string()
    .min(6, 'Password must contain at least 6 characters.')
    .required('Password is required.'),
});

export default function App() {
  const submitForm = (
    values,
    {
      resetForm,
    }
  ) => {
    Alert.alert(
```

```
    'Registration Successful',
    `Name: ${values.name}\nEmail: ${values.email}`
  );

  resetForm();
};

return (
  <KeyboardAvoidingView
    style={styles.container}
    behavior={
      Platform.OS === 'ios'
        ? 'padding'
        : undefined
    }
  >
    <Formik
      initialValues={{
        name: '',
        email: '',
```

```
    password: '',
  }}
  validationSchema={validationSchema}
  onSubmit={submitForm}
>
  {({
    values,
    errors,
    touched,
    handleChange,
    handleBlur,
    handleSubmit,
    isValid,
  }) => (
    <View style={styles.form}>
      <Text style={styles.heading}>
        Create Account
      </Text>

      <Text style={styles.label}>
```

```
  Name
</Text>

<TextInput
  style={[
    styles.input,
    touched.name &&
      errors.name &&
      styles.invalidInput,
  ]}
  placeholder="Enter your name"
  value={values.name}
  onChangeText={handleChange('name')}
  onBlur={handleBlur('name')}
/>

{touched.name && errors.name ? (
  <Text style={styles.errorText}>
    {errors.name}
  </Text>
```

```
) : null}


<Text style={styles.label}>
  Email
</Text>


<TextInput
  style={[
    styles.input,
    touched.email &&
      errors.email &&
      styles.invalidInput,
  ]}
  placeholder="Enter your email"
  value={values.email}
  onChangeText={handleChange('email')}
  onBlur={handleBlur('email')}
  keyboardType="email-address"
  autoCapitalize="none"
/>
```

```
{touched.email && errors.email ? (
  <Text style={styles.errorText}>
    {errors.email}
  </Text>
) : null}


<Text style={styles.label}>
  Password
</Text>


<TextInput
  style={[
    styles.input,
    touched.password &&
      errors.password &&
      styles.invalidInput,
  ]}
  placeholder="Enter your password"
  value={values.password}
```

```
  onChangeText={handleChange('password')}
  onBlur={handleBlur('password')}
  secureTextEntry
/>

{touched.password &&
errors.password ? (
  <Text style={styles.errorText}>
    {errors.password}
  </Text>
) : null}

<Pressable
  style={[
    styles.button,
    !isValid &&
      styles.disabledButton,
  ]}
  onPress={handleSubmit}
>
```

```
            <Text style={styles.buttonText}>
              Register
            </Text>
          </Pressable>
        </View>
      )}
    </Formik>
  </KeyboardAvoidingView>
 );
}

const styles = StyleSheet.create({
 container: {
  flex: 1,
  padding: 20,
  justifyContent: 'center',
  backgroundColor: '#eef2f7',
 },

 form: {
```

```
  padding: 22,
  backgroundColor: '#ffffff',
  borderRadius: 14,
  elevation: 4,
},

heading: {
  fontSize: 26,
  fontWeight: 'bold',
  textAlign: 'center',
  marginBottom: 24,
},

label: {
  fontSize: 15,
  fontWeight: '600',
  marginBottom: 6,
},

input: {
```

```
  height: 48,
  paddingHorizontal: 13,
  borderWidth: 1,
  borderColor: '#cbd5e1',
  borderRadius: 8,
},

invalidInput: {
  borderColor: '#dc2626',
},

errorText: {
  marginTop: 5,
  marginBottom: 12,
  fontSize: 13,
  color: '#dc2626',
},

button: {
  paddingVertical: 14,
```

```
    marginTop: 20,
    alignItems: 'center',
    backgroundColor: '#2563eb',
    borderRadius: 8,
  },

  disabledButton: {
    opacity: 0.6,
  },

  buttonText: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#ffffff',
  },
});
```

---

14. Firebase Email and Password

Authentication

The Firebase JavaScript SDK can be used with Expo Go for services such as Authentication, Firestore, Realtime Database, and Storage. Current Expo documentation requires a supported modern Firebase SDK.

Firebase uses modular functions such as:

- `initializeApp`
- `getAuth`
- `createUserWithEmailAndPassword`
- `signInWithEmailAndPassword`
- `signOut`
- `onAuthStateChanged`

These functions follow Firebase's current modular authentication API.

Step 1: Install Firebase

npx expo install firebase

Step 2: Configure Firebase

In the Firebase Console:

1. Create a Firebase project.
2. Add a web application to the project.
3. Copy the Firebase configuration.
4. Open Authentication.

5. Open Sign-in method.
6. Enable Email/Password authentication.

Step 3: Create `firebaseConfig.js`

Create this file in the project root:

```
import {
  getApp,
  getApps,
  initializeApp,
} from 'firebase/app';

import {
  getAuth,
} from 'firebase/auth';

const firebaseConfig = {
  apiKey: 'YOUR_API_KEY',
  authDomain: 'YOUR_PROJECT.firebaseapp.com',
  projectId: 'YOUR_PROJECT_ID',
  storageBucket: 'YOUR_PROJECT.appspot.com',
  messagingSenderId: 'YOUR_SENDER_ID',
```

```
  appId: 'YOUR_APP_ID',
};

const app =
  getApps().length === 0
    ? initializeApp(firebaseConfig)
    : getApp();

export const auth = getAuth(app);
```

Replace the sample values with the configuration provided by your Firebase project.

Step 4: Add Authentication Code to `App.js`

```
import React, {
  useEffect,
  useState,
} from 'react';

import {
```

```
  View,
  Text,
  TextInput,
  Pressable,
  Alert,
  ActivityIndicator,
  StyleSheet,
} from 'react-native';

import {
  createUserWithEmailAndPassword,
  onAuthStateChanged,
  signInWithEmailAndPassword,
  signOut,
} from 'firebase/auth';

import {
  auth,
} from './firebaseConfig';
```

```
export default function App() {
  const [email, setEmail] = useState('');
  const [password, setPassword] =
    useState('');

  const [user, setUser] = useState(null);
  const [screenLoading, setScreenLoading] =
    useState(true);

  const [buttonLoading, setButtonLoading] =
    useState(false);

  const [isRegisterMode, setIsRegisterMode] =
    useState(false);

  useEffect(() => {
    const unsubscribe = onAuthStateChanged(
      auth,
      (currentUser) => {
        setUser(currentUser);
```

```
      setScreenLoading(false);
    }
  );

  return unsubscribe;
}, []);

const validateForm = () => {
  if (!email.trim()) {
    Alert.alert(
      'Validation Error',
      'Email is required.'
    );

    return false;
  }

  if (password.length < 6) {
    Alert.alert(
      'Validation Error',
```

```
      'Password must contain at least 6 characters.'
    );

    return false;
  }

  return true;
};

const handleAuthentication = async () => {
  if (!validateForm()) {
    return;
  }

  try {
    setButtonLoading(true);

    if (isRegisterMode) {
      await createUserWithEmailAndPassword(
        auth,
```

```
      email.trim(),
      password
    );

    Alert.alert(
      'Success',
      'Your account has been created.'
    );
  } else {
    await signInWithEmailAndPassword(
      auth,
      email.trim(),
      password
    );
  }

  setEmail('');
  setPassword('');
} catch (error) {
  Alert.alert(
```

```
      'Authentication Error',
      error.message
    );
  } finally {
    setButtonLoading(false);
  }
};

const handleLogout = async () => {
  try {
    await signOut(auth);
  } catch (error) {
    Alert.alert(
      'Logout Error',
      error.message
    );
  }
};

if (screenLoading) {
```

```
  return (
    <View style={styles.centerContainer}>
      <ActivityIndicator size="large" />

      <Text style={styles.loadingText}>
        Checking authentication...
      </Text>
    </View>
  );
}

if (user) {
  return (
    <View style={styles.centerContainer}>
      <Text style={styles.heading}>
        Welcome
      </Text>

      <Text style={styles.userEmail}>
        {user.email}
```

```
      </Text>

      <Pressable
        style={styles.logoutButton}
        onPress={handleLogout}
      >
        <Text style={styles.buttonText}>
          Logout
        </Text>
      </Pressable>
    </View>
  );
}

return (
  <View style={styles.container}>
    <View style={styles.form}>
      <Text style={styles.heading}>
        {isRegisterMode
          ? 'Create Account'
```

```
    : 'Login'}
</Text>

<TextInput
  style={styles.input}
  placeholder="Enter email"
  value={email}
  onChangeText={setEmail}
  keyboardType="email-address"
  autoCapitalize="none"
/>

<TextInput
  style={styles.input}
  placeholder="Enter password"
  value={password}
  onChangeText={setPassword}
  secureTextEntry
/>
```

```
<Pressable
  style={styles.primaryButton}
  onPress={handleAuthentication}
  disabled={buttonLoading}
>
  {buttonLoading ? (
    <ActivityIndicator color="#ffffff" />
  ) : (
    <Text style={styles.buttonText}>
      {isRegisterMode
        ? 'Register'
        : 'Login'}
    </Text>
  )}
</Pressable>

<Pressable
  style={styles.switchButton}
  onPress={() =>
    setIsRegisterMode(
```

```
            (previousMode) => !previousMode
          )
        }
      >
        <Text style={styles.switchText}>
          {isRegisterMode
            ? 'Already have an account? Login'
            : 'New user? Create an account'}
        </Text>
      </Pressable>
    </View>
  </View>
 );
}


const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    justifyContent: 'center',
```

```
  backgroundColor: '#eef2f7',
},

centerContainer: {
  flex: 1,
  padding: 25,
  justifyContent: 'center',
  alignItems: 'center',
},

form: {
  padding: 22,
  backgroundColor: '#ffffff',
  borderRadius: 14,
  elevation: 4,
},

heading: {
  fontSize: 28,
  fontWeight: 'bold',
```

```
  textAlign: 'center',
  marginBottom: 25,
},

input: {
  height: 50,
  paddingHorizontal: 14,
  marginBottom: 16,
  borderWidth: 1,
  borderColor: '#cbd5e1',
  borderRadius: 8,
},

primaryButton: {
  minHeight: 50,
  justifyContent: 'center',
  alignItems: 'center',
  backgroundColor: '#2563eb',
  borderRadius: 8,
},
```

```
logoutButton: {
  paddingVertical: 14,
  paddingHorizontal: 35,
  marginTop: 25,
  backgroundColor: '#dc2626',
  borderRadius: 8,
},

switchButton: {
  padding: 15,
  marginTop: 10,
  alignItems: 'center',
},

buttonText: {
  fontSize: 16,
  fontWeight: 'bold',
  color: '#ffffff',
},
```

```
  switchText: {
    fontSize: 15,
    fontWeight: '600',
    color: '#2563eb',
  },

  userEmail: {
    fontSize: 18,
  },

  loadingText: {
    marginTop: 15,
    fontSize: 16,
  },
});
```

---

15. AsyncStorage for Local Storage

AsyncStorage is an asynchronous and persistent key-value storage system. It stores string values, so objects must be converted using `JSON.stringify()` and restored using `JSON.parse()`. The current API supports scoped storage instances created with `createAsyncStorage()`.

AsyncStorage is unencrypted, so sensitive information such as passwords should not be stored directly in it.

Installation

For Expo:

npx expo install @react-native-async-storage/async-storage

Complete Program

```
import React, {
  useEffect,
  useState,
} from 'react';


import {
```

```
  View,
  Text,
  TextInput,
  Pressable,
  Alert,
  StyleSheet,
} from 'react-native';

import {
  createAsyncStorage,
} from '@react-native-async-storage/async-storage';

const storage = createAsyncStorage(
  'studentApplication'
);

const PROFILE_KEY = 'studentProfile';

export default function App() {
  const [name, setName] = useState('');
```

```
const [savedName, setSavedName] =
  useState('');


const saveProfile = async () => {
  if (!name.trim()) {
    Alert.alert(
      'Validation Error',
      'Please enter a name.'
    );

    return;
  }

  try {
    const profile = {
      name: name.trim(),
      savedAt: new Date().toISOString(),
    };

    await storage.setItem(
```

```
      PROFILE_KEY,
      JSON.stringify(profile)
    );

    setSavedName(profile.name);
    setName('');

    Alert.alert(
      'Success',
      'The profile has been saved locally.'
    );
  } catch (error) {
    Alert.alert(
      'Storage Error',
      'Unable to save the profile.'
    );
  }
};

const loadProfile = async () => {
```

```
  try {
    const storedValue =
      await storage.getItem(PROFILE_KEY);

    if (storedValue) {
      const profile =
        JSON.parse(storedValue);

      setSavedName(profile.name);
    } else {
      setSavedName('');
    }
  } catch (error) {
    Alert.alert(
      'Storage Error',
      'Unable to load the profile.'
    );
  }
};
```

```
const removeProfile = async () => {
  try {
    await storage.removeItem(PROFILE_KEY);

    setSavedName('');

    Alert.alert(
      'Removed',
      'The saved profile has been removed.'
    );
  } catch (error) {
    Alert.alert(
      'Storage Error',
      'Unable to remove the profile.'
    );
  }
};

useEffect(() => {
  loadProfile();
```

```
}, []);


return (
  <View style={styles.container}>
    <Text style={styles.heading}>
      AsyncStorage Example
    </Text>

    <View style={styles.card}>
      <Text style={styles.label}>
        Enter Student Name
      </Text>

      <TextInput
        style={styles.input}
        placeholder="Enter name"
        value={name}
        onChangeText={setName}
      />
```

```
<Pressable
  style={styles.saveButton}
  onPress={saveProfile}
>
  <Text style={styles.buttonText}>
    Save Name
  </Text>
</Pressable>

<Pressable
  style={styles.loadButton}
  onPress={loadProfile}
>
  <Text style={styles.buttonText}>
    Load Saved Name
  </Text>
</Pressable>

<Pressable
  style={styles.removeButton}
```

```
        onPress={removeProfile}
      >
        <Text style={styles.buttonText}>
          Remove Saved Name
        </Text>
      </Pressable>
    </View>

    <View style={styles.resultCard}>
      <Text style={styles.resultLabel}>
        Saved Name
      </Text>

      <Text style={styles.resultValue}>
        {savedName || 'No name is currently saved.'}
      </Text>
    </View>
  </View>
 );
}
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    paddingTop: 60,
    backgroundColor: '#f1f5f9',
  },

  heading: {
    fontSize: 28,
    fontWeight: 'bold',
    textAlign: 'center',
    marginBottom: 25,
  },

  card: {
    padding: 20,
    backgroundColor: '#ffffff',
    borderRadius: 12,
```

```
  elevation: 3,
},

label: {
  fontSize: 16,
  fontWeight: '600',
  marginBottom: 8,
},

input: {
  height: 50,
  paddingHorizontal: 14,
  marginBottom: 18,
  borderWidth: 1,
  borderColor: '#cbd5e1',
  borderRadius: 8,
},

saveButton: {
  padding: 14,
```

```
  marginBottom: 12,
  alignItems: 'center',
  backgroundColor: '#16a34a',
  borderRadius: 8,
},

loadButton: {
  padding: 14,
  marginBottom: 12,
  alignItems: 'center',
  backgroundColor: '#2563eb',
  borderRadius: 8,
},

removeButton: {
  padding: 14,
  alignItems: 'center',
  backgroundColor: '#dc2626',
  borderRadius: 8,
},
```

```
buttonText: {
  fontSize: 15,
  fontWeight: 'bold',
  color: '#ffffff',
},

resultCard: {
  padding: 20,
  marginTop: 20,
  backgroundColor: '#ffffff',
  borderRadius: 12,
},

resultLabel: {
  fontSize: 15,
  fontWeight: 'bold',
  marginBottom: 8,
},
```

```
  resultValue: {
    fontSize: 19,
  },
});
```

Recommended Learning Sequence

Build and practise the applications in the following order:

1. Core UI components
2. StyleSheet and Flexbox
3. User input
4. Props
5. State
6. Form validation
7. Stack navigation
8. Tab navigation
9. Passing data between screens
10. Context API
11. API integration
12. AsyncStorage
13. Firebase Authentication

Do not try to build all features in one application initially. Complete and test

each concept separately before combining them into a larger project.